

العمليات الحسابية

الأعداد شائعة جدًا في البرمجة، إذ تُستخدم لتمثيل مختلف القيم، مثل أبعاد حجم الشاشة، والمواقع الجغرافية، والمبالغ المالية، ومقدار الوقت الذي مر منذ بداية فيديو، والألوان وغير ذلك. تعد القدرة على تنفيذ العمليات الرياضية بفعالية في البرمجة مهارة مهمة، لأنك الأعداد ستكون في متناول الأيدي دومًا. الفهم الجيد للرياضيات يمكن أن يساعدك على أن تصبح مبرمجًا أفضل، إلا أنه ليس شرطًا أساسيًا. فالرياضيات أداة لتحقيق ما ترغب في تحقيقه، وطريقة لتحسين آلية التنفيذ.

سنعمل مع أكثر نوعي البيانات استخدامًا في بايثون، وهما الأعداد الصحيحة والأعداد العشرية:

- الأعداد الصحيحة هي أعداد كاملة يمكن أن تكون موجبة أو سالبة أو معدومة مثل (...، -1، 0، 1، ...).
- الأعداد العشرية هي أعداد حقيقية تحتوي على فاصلة عشرية (كما في 9.0 أو -2.25).

سنلقي في هذا الفصل نظرةً على العوامل (operators) التي يمكن استخدامها مع أنواع البيانات العددية في بايثون.

1. العوامل

العامل (operator) هو رمز أو دالة تمثل عمليةً حسابية، إذ جاءت تسمية عامل من عملية، أي العامل الذي يجري عملية. على سبيل المثال، في الرياضيات، علامة الجمع أو + هي العامل الذي يجري عملية الجمع.

في بايثون، سنرى بعض العوامل المألوفة، والتي استُعيِرت من الرياضيات، لكن هناك عوامل أخرى خاصة بمجال البرمجة.

الجدول التالي مرجعٌ سريعٌ للعوامل الحسابية في بايثون. سنغطي جميع هذه العمليات في هذا الفصل.

العملية	الناتج
$x + y$	مجموع x و y
$x - y$	طرح x من y
$-x$	عكس إشارة x
$+x$	نفس قيمة x
$x * y$	ضرب x بـ y
x / y	قسمة x على y
$x // y$	حاصل القسمة التحتية لـ x على y
$x \% y$	باقي قسمة x على y
$x ** y$	x أس y

سنحدث أيضًا عن عوامل الإسناد المركبة (compound assignment operators)، بما

في ذلك $+=$ و $*$ ، التي تجمع عاملاً حسابيًا مع العامل $=$.

2. الجمع والطرح

في بايثون، يعمل معاملا الجمع والطرح مثل ما هو معروف في الرياضيات. في الواقع، يمكنك استخدام لغة بايثون آلة حاسبة. لنلق نظرة على بعض الأمثلة، بدءًا من الأعداد الصحيحة:

```
print(1 + 5)
```

والناتج:

```
6
```

بدلاً من تمرير أعداد صحيحة مباشرة إلى الدالة `print`، يمكننا تهيئة المتغيرات بأعداد صحيحة:

```
a = 88
b = 103

print(a + b)
```

وسينتج لنا:

```
191
```

الأعداد الصحيحة يمكن أن تكون موجبة أو سالبة (أو معدومة أيضاً)، لذلك يمكننا إضافة عدد سالب إلى عدد موجب:

```
c = -36
d = 25

print(c + d) # -11
```

الجمع سيكون مشابهًا مع الأعداد العشرية:

```
e = 5.5
f = 2.5

print(e + f) # 8.0
```

إذا جمعنا عددين عشريين معًا، ستعيد بايثون عددًا عشريًا.

صياغة الطرح تشبه صياغة الجمع، ما عدا أنك ستستبدل بعامل الطرح (-)

عامل الجمع (+):

```
g = 75.67
h = 32

print(g - h) # 43.67
```

هنا، طرحنا عددًا صحيحًا من عدد عشري. ستعيد بايثون عددًا عشريًا إذا كان أحد الأعداد

المتضمنة في المعادلة عشريًا.

3. العمليات الحسابية الأحادية

يتكون التعبير الرياضي الأحادي (unary mathematical expression) من مكون أو عنصر

واحد فقط، ويمكن في بايثون استخدام العلامتين + و - بمفردهما عبر قرنهما بقيمة لإعادة

القيمة نفسها (عبر العامل +)، أو تغيير إشارة القيمة (عبر العامل -).

تشير علامة الجمع - رغم أنّها لا تُستخدم كثيرًا - إلى هوية القيمة (identity of the value)،

أي تعيد القيمة نفسها. يمكننا استخدام علامة الجمع مع القيم الموجبة:

```
i = 3.3
print(+i) # 3.3
```

عندما نستخدم علامة الجمع مع قيمة سالبة، فسُتُعَاد القيمة نفسها، وفي هذه الحالة ستكون

قيمة سالبة أيضًا:

```
j = -19
print(+j)    # -19
```

علامة الطرح، على خلاف علامة الجمع، تغيّر إشارة القيمة. لذلك، عندما نضعها مع قيمة

موجبة، سَتُعَاد القيمة السالبة منها:

```
i = 3.3
print(-i)    # -3.3
```

بالمقابل، عندما نستخدم عامل الطرح الأحادي (minus sign unary operator) مع قيمة

سالبة، فسُتُعَاد القيمة الموجبة منها:

```
j = -19
print(-j)    # 19
```

سُتُعِيد العمليتان الحسابيتان الأحاديتان + و - إمّا هوية القيمة المعطاة، أو القيمة المعاكسة

في الإشارة للقيمة المعطاة على التوالي.

4. الضرب والقسمة

مثل الجمع والطرح، الضرب والقسمة في بايثون مشابهان لما هو معروف في الرياضيات.

علامة الضرب في بايثون هي *، وعلامة القسمة هي /.

فيما يلي مثال على ضرب عددين عشريين في بايثون:

```
k = 100.1
```

```
l = 10.1
```

```
print(k * l)    # 1011.0099999999999
```

عندما تُجرى عملية القسمة في بايثون 3، فسيكون العدد المُعاد دائماً عشرياً، حتى لو

استخدمت عددين صحيحين:

```
m = 80
```

```
n = 5
```

```
print(m / n)    # 16.0
```

هذا أحد الاختلافات الرئيسية بين بايثون 2 و بايثون 3. الإجابة في بايثون 3 تكون كسرية،

فعند استخدام / لتقسيم 11 على 2 مثلاً، فستُعاد القيمة 5.5. أمّا في بايثون 2، فحاصل التعبير 11/2 هو 5.

يُجري العامل / في بايثون 2 قسمة عملية القسمة مع تقريب الناتج إلى أصغر عدد صحيح

له (هذه العملية تدعى القسمة التقريبية [floor division])، إذ أنّه إن كان حاصل القسمة يساوي x، فسيكون ناتج عملية القسمة في بايثون 2 أكبر عدد من الأعداد الصحيحة الأصغر من x. أو تساوي x. إذا نُفّذت المثال `print(80 / 5)` أعلاه في بايثون 2 بدلاً من بايثون 3، فسيكون الناتج هو 16، دون الجزء العشري.

في بايثون 3، يمكنك استخدام العامل // لإجراء القسمة التقريبية. التعبير `40 // 100`

سيعيد القيمة 2. القسمة التحتية مفيدة في حال كنت تريد أن يكون حاصل القسمة عدداً صحيحاً.

5. عامل باقي القسمة (Modulo)

العامل % هو باقي القسمة (modulo)، والذي يُرجع باقي عملية القسمة. هذا مفيد للعثور على الأعداد التي هي مضاعفات لنفس العدد. المثال التالي يوضح كيفية استخدام عامل الباقي:

```
o = 85
p = 15

print(o % p)    # 10
```

حاصل قسمة 85 على 15 هو 5، والباقي 10. القيمة 10 هي التي ستُعاد هنا لأنَّ عامل

الباقي يعيد باقي عملية القسمة.

إذا استخدمنا عددين عشريين مع عامل الباقي، فسيُعاد عدد عشري:

```
q = 36.0
r = 6.0

print(o % p)    # 0.0
```

في حال قسمة 36.0 على 6.0، فلن يكون هناك باقٍ، لذلك تعاد القيمة 0.0.

6. القوة (Power)

يُستخدم عامل القوة ** (يقال له أحياناً «الأس») في بايثون لرفع العدد الأيسر لقوة الأس

للعدد الأيمن. وهذا يعني أنه في التعبير 3 ** 5، العدد 5 سيُرفَع إلى القوة 3. في الرياضيات،

غالبًا ما نرى هذا التعبير يُكتب على الشكل 5³، إذ يُضرب العدد 5 في نفسه 3 مرات. في بايثون،

التعبيران 3 ** 5 و 5 * 5 * 5 سيعطيان النتيجة نفسها.

سنستخدم في المثال التالي المتغيرات:


```
s = 52.25
t = 7

print(s ** t)      # 1063173305051.292
```

رفع العدد العشري 52.25 إلى القوة 7 عبر عامل الأس `**` سينتج عنه عدد عشري كبير.

7. أسبقية العمليات الحسابية

في بايثون، كما هو الحال في الرياضيات، علينا أن نضع في حساباتنا أن المعاملات ستُقيم وفقاً لنظام الأسبقية، وليس من اليسار إلى اليمين، أو من اليمين إلى اليسار. إذا نظرنا إلى التعبير التالي:

```
u = 10 + 10 * 5
```

قد نقرأه من اليسار إلى اليمين، ولكن تذكّر أنّ عملية الضرب ستُجرى أولاً، لذا إن استدعينا `print(u)`، فسنحصل على القيمة التالية:

```
60
```

هذا لأن `10 * 5` ستُقيم أولاً، وسينتج عنها العدد 50، ثم يضاف إليها العدد 10 لنحصل على 60 في النهاية.

إذا أردنا بدلاً من ذلك إضافة القيمة 10 إلى 10، ثم ضرب المجموع في 5، فيمكننا استخدام الأقواس كما نفعل في الرياضيات تمامًا:

```
u = (10 + 10) * 5
print(u)      # 100
```

إحدى الطرق البسيطة لتذكر الأسبقيات هي حفظ البيتين التاليين لتذكر أوائل كلماتهما:

قم أبعد ضيقًا ... قم جد طريقًا

الأسبقية	العملية	الحرف
1	القوس	قم
2	الأس	أبعد
3	الضرب	ضيقًا
4	القسمة	قم
5	الجمع	جد
6	الطرح	طريقًا

8. عامل الإسناد (Assignment Operators)

أكثر عامل الإسناد استخدامًا هو إشارة التساوي =. يُسند عامل الإسناد (أو عامل التعيين) =

القيمة الموجودة على يمينه إلى المتغير الموضوع على يساره. على سبيل المثال، تُسند

التعليمة $v = 23$ العدد الصحيح 23 للمتغير v .

من الشائع استخدام عاملات الإسناد المركبة التي تجري عملية رياضية على قيمة المتغير،

ثم تُسند القيمة الجديدة الناتجة إلى ذلك المتغير. تجمع عوامل الإسناد المركبة بين عامل رياضي

والعامل =؛ ففي حال الجمع، نستخدم + مع = للحصول على عامل الإسناد المركب +=. لنطبّق

ذلك في مثال عملي:

```
w = 5
w += 1
print(w)    # 6
```

أولاً، تُسند القيمة 5 إلى المتغير `w`، ثم نستخدم معامل الإسناد المركب `+=` لإضافة العدد الصحيح إلى قيمة المتغير الأيسر، ثم تُسند النتيجة إلى المتغير `w`.
تُستخدم معاملات الإسناد المركبة استخدامًا متكررًا مع **حلقات for التكرارية**، والتي ستستخدمها عندما تريد تكرار عملية عدة مرات:

```
for x in range (0, 7):
    x *= 2
    print(x)
```

والناتج سيكون:

```
0
2
4
6
8
10
12
```

تمكنا باستخدام الحلقة **for** من أتمتة العملية `*` التي تضرب قيمة المتغير `w` بالعدد 2، ثم تُسند النتيجة إلى المتغير `w` لأجل استخدامها في التكرار التالي من الحلقة.
لدى بايثون معامل إسناد مركب مقابل لكل من المعاملات الحسابية التي تطرقنا آنفًا إليها وهي:

```

y += 1    # إضافة القيمة ثم إسنادها
y -= 1    # طرح القيمة ثم إسنادها
y *= 2    # ضرب القيمة ثم إسنادها
y /= 3    # تقسيم القيمة ثم إسنادها
y // = 5   # تقسيم سفلي القيمة ثم إسنادها
y **= 2   # تنفيذ عامل الأس على القيمة ثم إسنادها
y %= 3    # إعادة باقي قسمة القيمة ثم إسنادها

```

يمكن أن يكون عامل الإسناد المركب مفيدًا عندما تحتاج إلى الزيادة أو الإنقاص التدريجي، أو عندما تحتاج إلى أتمتة عمليات معينة في برنامجك.

9. إجراء العمليات الرياضية عبر الدوال

بعد أن تعرفنا كيفية إجراء العمليات الرياضية عبر العوامل (operators)، سنستعرض في هذا القسم بعض الدوال الرياضية المُضمَّنة التي يمكن استخدامها مع أنواع البيانات العددية في بايثون 3. فتتضمَّن بايثون 3 العديد من الدوال التي يمكنك استخدامها بسهولة في أي برنامج. تتيح لك بعض تلك الدوال تحويل أنواع البيانات، والبعض الآخر خاص بنوع معين، مثل السلاسل النصية.

سنلقي نظرة على الدوال التالية:

- `abs()`: للحصول على القيمة المطلقة
- `divmod()`: للحصول على الحاصل والباقي في وقت واحد
- `pow()`: لرفع عدد لقوة معينة
- `round()`: لتقريب عدد بمنازل عشرية محددة
- `sum()`: لحساب مجموع العناصر في كائن قابلٍ للتكرار (iterable)

فهم هذه الدوال سيتمنحك مرونة أكبر في التعامل مع البيانات العددية، ويساعدك على اتخاذ قرارات مدروسة عند تحديد العوامل والدوال التي عليك استخدامها.

١. القيمة المطلقة

تعيد الدالة المضمّنة `abs()` القيمة المطلقة للعدد الذي يُمرر إليها. في الرياضيات، تشير القيمة المطلقة إلى العدد نفسه إن كانت القيمة موجبة، أو القيمة المعاكسة إن كانت القيمة سالبة. مثلاً، القيمة المطلقة للعدد 15 هي 15، والقيمة المطلقة للعدد -74 هي 74، والقيمة المطلقة للعدد 0 هي 0.

القيمة المطلقة مفهوم مهم في الحساب والتحليل، كما أنَّها مفيدة كذلك في المواقف اليومية، مثل حساب المسافة المقطوعة. على سبيل المثال، إذا كنت تحاول الوصول إلى مكان يبعد 58 ميلاً، ولكنتك تجاوزت ذلك المكان، وسافرت 93 **فرسجاً**. فإنَّ حسبت عدد الفراسخ التي ينبغي أن تقطعها الآن للوصول إلى الوجهة المقصودة، فسوف ينتهي بك المطاف بعدد سالب، لكن لا يمكنك السفر عدداً سالباً من الفراسخ!

سنستخدم الدالة `abs()` لحل هذه المشكلة:

```
# عدد الفراسخ التي تفصلنا عن الوجهة انطلاقاً من المُنطلق
miles_from_origin = 58

# الفراسخ المقطوعة من المُنطلق إلى الوجهة
miles_travelled = 93

# حساب عدد الفراسخ من الموقع الحالي
miles_to_go = miles_from_origin - miles_travelled

# طباعة عدد الفراسخ المتبقية بعدد سالب
```

```
print(miles_to_go)
# حساب القيمة المطلقة للعدد السالب
print(abs(miles_to_go))
```

المخرجات ستكون:

```
-35
35
```

لولا استخدام الدالة `abs()`، لحصلنا على عدد سالب، أي -35. ورغم أنَّ `miles_travelled` أصغر من `miles_from_origin`، فإنَّ الدالة `abs()` تحل إشكالية العدد السالب. عندما نمُرُّ لها عددًا سالبًا، ستعيد الدالة `abs()` عددًا موجبًا، لأنَّ القيمة المطلقة دائمًا تعيد أعدادًا موجبة أو معدومة.

في المثال التالي، سنمرر للدالة `abs()` عددًا موجبًا، وكذلك الصفر:

```
print(abs(89.9)) # 89.9
print(abs(0)) # 0
```

ب. العثور على الحاصل والباقي بدالة واحدة

القسمة التقريبية (`floor division`)، التي تُعيد حاصل القسمة `[quotient]` مع تقريبه)، وقسمة الباقي (`modulo division`)، التي تعيد باقي القسمة `[remainder]`، مرتبطان ارتباطًا وثيقًا، وقد يكون من المفيد استخدام دالة تجمع بين العمليتين معًا. تجمع الدالة المضَمَّنة `divmod()` بين العمليتين، إذ تعيد أولاً حاصل عملية القسمة التقريبية، ثم الباقي. ينبغي تمرير عددين إلى الدالة `divmod()`، على النحو التالي:

```
divmod(a,b)
```

تكافئ هذه الدالة العمليتين التاليتين:

```
a // b
a & b
```

لنفترض أننا كتبنا كتابًا يحتوي 80 ألف كلمة. يريد الناشر أن تحتوي كل صفحة من الكتاب ما بين 300 و 250 كلمة، ونود أن نعرف عدد الصفحات التي ستشكل الكتاب بحسب عدد كلمات الصفحة الذي اخترناه. باستخدام الدالة `divmod()` يمكننا أن نعرف على الفور عدد الصفحات في الكتاب، وعدد الكلمات المتبقية التي سننقل إلى صفحة إضافية.

```
# كم عدد الكلمات في كتابنا
words = 80000

# الخيار A: عدد كلمات الصفحة هو 300
per_page_A = 300

# الخيار B: عدد كلمات الصفحة هو 250
per_page_B = 250

print(divmod(words, per_page_A)) # حساب الخيار A
print(divmod(words, per_page_B)) # حساب الخيار B
```

وسينتج عن هذه الشيفرة:

```
(266, 200)
(320, 0)
```

في الخيار A، سنحصل على 266 صفحة مليئة بالكلمات، و 200 كلمة متبقية (ثلاث صفحات)، والمجموع هو 267 صفحة، وفي الخيار B، سنحصل على كتاب من 320 صفحة. إن أردنا الحفاظ

على البيئة، فالخيار A قد يكون أفضل، ولكن إذا أردنا تصميمًا جذابًا، أو الكتابة بحجم خط كبير، فقد نختار الخيار "B".

تقبل الدالة `divmod()` الأعداد الصحيحة والأعداد العشرية، في المثال التالي سُمِّر عددًا عشريًا إلى الدالة `divmod()`:

```
a = 985.5
b = 115.25

print(divmod(a,b)) # (8.0, 63.5)
```

في هذا المثال، العدد 8.0 هو حاصل القسمة التحتية للعدد 985.5 مقسومًا على 115.25، و 63.5 هو الباقي.

يمكنك استخدام عامل القسمة التحتية `//` و عامل الباقي `%` للتحقق من نتيجة `divmod()`:

```
print(a//b) # 8.0
print(a%b) # 63.5
```

ج. القوة (Power)

في بايثون، يمكنك استخدام عامل القوة `**` (أو الأس) لرفع عدد إلى قوة معينة، أو يمكنك استخدام الدالة `pow()` المضمَّنة التي تأخذ عددين وتجري العملية نفسها.

لتوضيح كيفية عمل الدالة `pow()`، لنقل أننا نجري أبحاثًا على البكتيريا، ونريد أن نقدّر عدد البكتيريا التي سنحصل عليها في نهاية اليوم إذا بدأنا ببكتيريا واحدة.

لنفترض أن البكتيريا التي نعمل عليها تتضاعف في كل ساعة، لذلك النتيجة النهائية ستكون 2 قوة العدد الكلي لعدد الساعات التي مرت (24 في حالتنا).


```
hours = 24
total_bacteria = pow(2, hours)

print(total_bacteria)    # 16777216
```

لقد مررنا عددين صحيحين للدالة `pow()`، والنتيجة التي حصلنا عليها، والتي تمثل عدد البكتيريا بحلول نهاية اليوم، هي أكثر من 16 مليون بكتيريا. في الرياضيات، نكتب "3 أس 3" بشكل عام على الشكل 3^3 والتي تكافئ $3 \times 3 \times 3$. أي 27. ولحساب 3^3 في بايثون، نكتب `pow(3,3)`، إذ تقبل الدالة `pow()` الأعداد الصحيحة والأعداد العشرية، وتوفّر بديلاً لعامل الأس `**`.

د. تقريب الأعداد

تقريب الأعداد (Rounding Numbers) ضروري عند العمل مع الأعداد العشرية التي تحتوي على الكثير من المنازل (الأجزاء) العشرية. تقبل الدالة المضمنة `round()` عددين: أحدها يمثل العدد المراد تقريبه والآخر يحدّد عدد المنازل العشرية المراد الإبقاء عليها (أي قيمة التقريب). سنستخدم هذه الدالة لتقريب عدد عشري له أكثر من 10 منازل عشرية والحصول على عدد بأربعة منازل عشرية فقط:

```
i = 17.34989436516001
print(round(i,4))    # 17.3499
```

في المثال أعلاه، تم تقريب العدد 17.34989436516001 إلى 17.3499 لأننا حدّدنا عدد المنازل العشرية التي ينبغي الاختصار عليها بأربعة.

لاحظ أيضًا أنَّ الدالة `round()` تقرّب الأعداد إلى الأعلى، لذا بدلاً من إعادة 17.3498، فقد أعادت 17.3499، لأنَّ الرقم الذي يلي المنزل العشري 8 هو الرقم 9. وسيقرّب أي عدد متبوع بالعدد 5 أو أكبر إلى العدد الصحيح التالي.

في الحياة اليومية، تُقرّب الأعداد لأسباب كثيرة، وخاصة في التعاملات المائيّة؛ فلا يمكننا تقسيم فلس واحد بالتساوي بين عدة أصدقاء مثلاً.

سنكتب في المثال التالي برنامجاً بسيطاً يمكنه حساب البقشيش، إذ سنحدّد فيه قيم المتغيرات، ولكن يمكنك إعادة كتابة البرنامج لجعل المستخدمين يدخلون القيم بأنفسهم. في هذا المثال، ذهب 3 أصدقاء إلى مطعم، وأرادوا تقسيم الفاتورة، والتي تبلغ 87.93 دولارًا بالتساوي، بالإضافة إلى إكرامية (بقشيش) بنسبة 20%:

```
bill = 87.93          # إجمالي الفاتورة
tip = 0.2             # بقشيش 20 %
split = 3             # عدد الناس الذين سيتشاركون الفاتورة

# حساب الفاتورة الإجمالية
total = bill + (bill * tip)
# حساب ما ينبغي أن يدفعه كل شخص
each_pay = total / split
# ما ينبغي أن يدفعه كل شخص قبل التقريب
print(each_pay)
# تقريب العدد فلا يمكننا تقسيم الفلسات
print(round(each_pay, 2))
```

والمخرجات ستكون:

```
35.172000000000004
35.17
```

في هذا البرنامج، نطلب أولاً إخراج العدد بعد حساب إجمالي الفاتورة والإكراميات مقسوماً على 3، النتيجة ستكون عدداً يتضمّن الكثير من المنازل العشرية: 35.172000000000004. نظراً لأنّ هذا العدد لا يمثّل مبلغاً مالياً واقعياً، فإنّنا نستخدم الدالة `round()` ونقرّب المنازل العشرية على 2، حتى نتمكّن من توفير ناتج يمكن للأصدقاء الثلاثة أن يدفعوه: 35.17. إذا كنت تفضل التقريب إلى عدد بلا منازل عشرية، يمكنك تمرير 0 كمعامل ثانٍ إلى الدالة `round()`:

```
round(345.9874590348545304636, 0)
```

القيمة الناتجة ستكون 346.0.

يمكنك أيضاً تمرير الأعداد الصحيحة إلى `round()` دون الخوف من تلقي خطأ، وهذا مفيد في حال تلقيت من المستخدم عدداً صحيحاً بدلاً من عدد عشري. وفي هذه الحالة، سيُعاد عدد صحيح.

هـ. حساب المجموع

تُستخدم الدالة `sum()` لحساب مجاميع أنواع البيانات العددية المركّبة (numeric compound data types)، بما في ذلك القوائم، والصفوف، والقواميس.

يمكننا تمرير قائمة إلى الدالة `sum()` لجمع كل عناصرها بالترتيب من اليسار إلى اليمين:

```
some_floats = [1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7, 8.8, 9.9]
print(sum(some_floats))    # 49.5
```

نفس النتيجة سنحصل عليها إن استخدمنا الصفوف والقواميس:

```
# حساب مجموع الأعداد في الصف
print(sum((8,16,64,512)))
# حساب مجموع الأعداد في القاموس
print(sum({-10: 'x', -20: 'y', -30: 'z'}))
```

المخرجات:

```
60
-60
```

يمكن أن تأخذ الدالة `sum()` وسيطين، الوسيط الثاني سيُضاف إلى المجموع الناتج:

```
some_floats = [1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7, 8.8, 9.9]

print(sum(some_floats, 0.5))          # 50.0
print(sum({-10: 'x', -20: 'y', -30: 'z'},60)) # 0
```

القيمة الافتراضية للوسيط الثاني هي 0.

10. خلاصة الفصل

غطّينا في هذا الفصل العديد من العوامل والدوال التي ستستخدمها مع الأعداد الصحيحة والعشرية لإجراء أهم العمليات الرياضية ولا زال هنالك الكثير منها، فننصحك بزيارة توثيق العمليات العددية في بايثون وتوثيق الأنواع العددية في بايثون في موسوعة حسوب لمزيد من التفاصيل.